

Range Minimum Queries

Feyzi Eren Gündoğdu

Karlsruhe Institute of Technology, Germany

1 Introduction

This report presents a comparison of different data structures and their performance for Range Minimum Queries (RMQ) in C++.

Six different data structures, including two naive and four advanced approaches, are discussed in this report. The naive algorithms, *OnTheFlyNaive* and *PrecomputedNaive*, are included to provide upper-bound performance comparisons for RMQ. The advanced data structures considered are *SparseTable*, *SegmentTree*, *BlockSparseTable*, and *CartesianTree*.

For some of the advanced data structures, different variants were evaluated and are represented by different suffixes (e.g., *BST16* denotes a Block Sparse Table with a block size of 16). n represents the input size, B represents block size.

TODO FILES

2 Methods

In this section, each data structure is discussed in detail. Only the non-naive aspects of the implementation and analysis are presented.

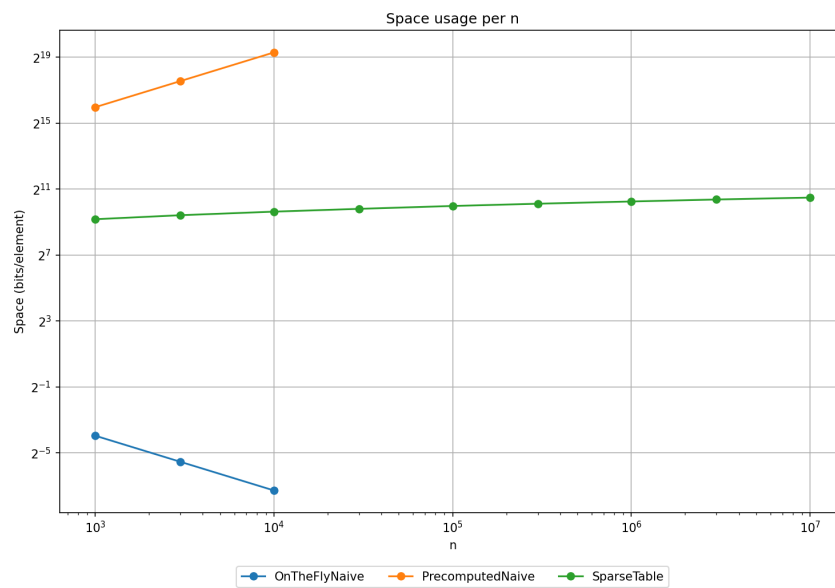
2.1 OnTheFlyNaive

This data structure is simply an array that stores the data directly and answers queries by computing the result on the fly. Due to this straightforward approach, it requires $O(1)$ space overhead and requires $O(n)$ time to answer each query, as can be seen in Figure 1 and Figure 2.

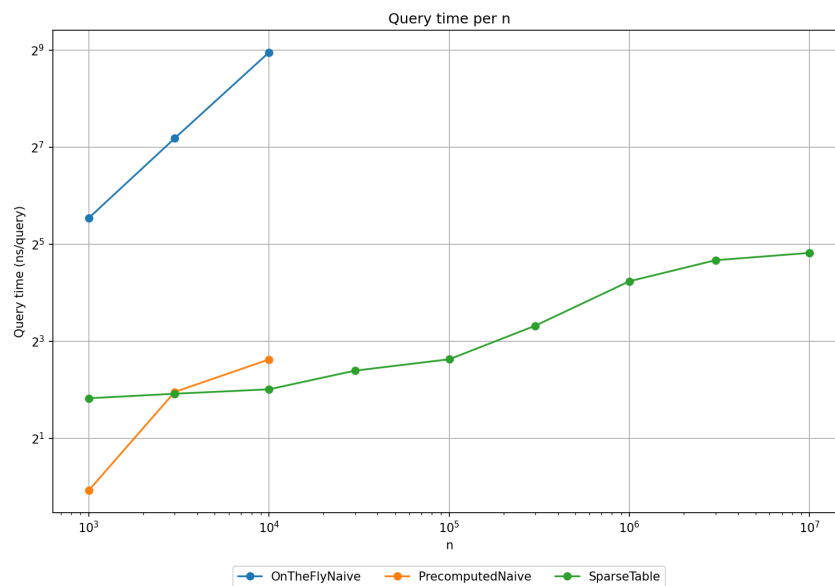
3 Results

TODO

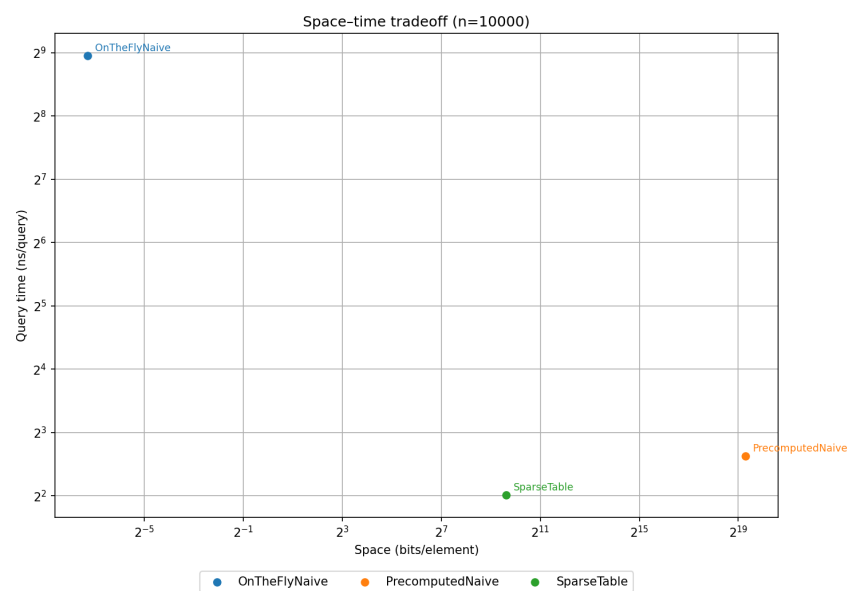
2 Range Minimum Queries



■ Figure 1 overhead space usage



■ Figure 2 query time



■ **Figure 3** space-time trade-off